

Combined statistics	
Details being fetched from Warehouse1, Warehouse2 and Corporate HQ (Web) online	
Select Department :	Department ID: 00010000056924
Total Items 14	
[1] ID: 000100000284721 Alias Key Operations Department, Prague DELIVERY DATE TO WAREHOUSE1: 2013-03-26 14:00:40.456 (DELIVERED) DELIVERY DATE TO WAREHOUSE2: 2013-03-15 14:30:32.0 (DELIVERED) DELIVERY DATE TO WAREHOUSE3: 15.03.2013	
[2] ID: 000100000279379 Richard Screwdriver Operations Department, Prague DELIVERY DATE TO WAREHOUSE1: 2013-03-26 14:44:42.652 (DELIVERED) DELIVERY DATE TO WAREHOUSE2: 2013-03-26 15:44:32.0 (DELIVERED) DELIVERY DATE TO WAREHOUSE3: 26.03.2013	
[3] ID: 000100000282384 Phillips Screwdriver Operations Department, Prague DELIVERY DATE TO WAREHOUSE1: 2013-03-15 13:59:02.77 (DELIVERED) DELIVERY DATE TO WAREHOUSE2: 2013-03-26 15:58:54.0 (BROKEN) DELIVERY DATE TO WAREHOUSE3: 15.03.2013	
[4] ID: 00010000059645 Jodifier rad Operations Department, Prague DELIVERY DATE TO WAREHOUSE1: 2013-08-22 10:01:32.044 (DELIVERED) DELIVERY DATE TO WAREHOUSE2: 2013-08-22 19:59:51.0 (DELIVERED) DELIVERY DATE TO WAREHOUSE3: 22.08.2013	

Figure 2: Two warehouses stats

```

15.     int exitVal = p.waitFor();
16.     System.out.println( "Exited with error code " +
17.     exitVal );
18. } catch (Exception e)
19. {
20.     e.printStackTrace();
21.     System.out.println(e.getMessage());
22. }
23. <br>DELIVERY DATE TO WAREHOUSE3: <c:out value="%<= hqDate
24. %>"%N/A/c:c:out>

```

Listing 5. This Java code implicitly calls the shell-script and transforms its output into JSTL notation.

At Line 1, we declare a JSTL variable called *ROWID*, the value for which is transferred during a big cycle from the corresponding variable *row_level.ID*. Such transformation is necessary for a more flexible string operation within the page context and the following Java-context.

Between Lines 2 and 22 there's a Java code that is directly responsible for handling the logic of processing a final date from the HTML page by means of the shell-script. For example, in Line 3, we convert a string variable from the JSP-page to use it later in 'pure' Java code. For debugging purposes only, in the following Line 4 we display that transformed value for standard output—in the future, you're welcome to comment this line out. Next, Line 5 prepares the environment string with all parameters, so the *Runtime* process could be executed. This method itself is enclosed in the *try {}* section—look at Lines 6-17. Please note that in order to handle exceptions, it is followed by *catch {}* section as well—Lines 17-21.

Let's now look at how we should start the *Runtime* process.

With the help of the already prepared 's' string that contains environment variables, we declare and initialise the new process—"Process *p* = *Runtime.getRuntime().exec(s);*". In order to be able to read what our shell-script spewed, we use *BufferedReader* (Line 8). After while cycle (Lines 10-14) we'll get the last line generated by *get_hqdate.sh*. This line is saved into the second variable *hqDate*. Lines 9 and 16, as well as Line 4, are considered for debugging purposes only. They serve for initial debugging and later it is advised to comment them out. Or, if you choose to, leave it as is—it will allow you to see online (by using '*tail -f catalina.out*' at the server side) how popular your new statistics are among your users. The process '*p*' must successfully finish its execution (Lines 15-16). And, if there's no error, you'll see inside the *catalina.out* file an exit code with value '0'. That's a perfect job.

Our processing is signalled by JSTL-function *<c:out>* – it outputs a value from the Java variable *hqDate* and prints at the JSP-page.

This large code block must be inserted between Lines 64 and 65 of the previous code (which was discussed in detail in the article 'Connect All Data Sources Effectively, Part 1' – OSFY, July 2013). Then you need to set up a fetching mechanism by means of the *wget* or *fetch* commands—after which you will have a local copy of an exported HTML page in the necessary format from HQ website (see Listing 1). Proceed further and check in your Web browser how all the data fits together. If everything is correct, you'll see how perfectly data from different sources is combined on one page (see Figure 2).

As you can see, the process of mixing different data is made simple. When you have a unique ID, and the accompanying details around this ID, you're free to combine those details as you wish. What seems more fascinating is the fact that all such data can be stored in various different formats: be it casual SQL-storage accessed by means of a regular JDBC-driver or in the form of either an HTML page, plain CVS, or an XLS/ODS spreadsheet. The latter seems even more interesting, because you have to deal with various XML-parsers. Compare that with our rather amateurish shell-script named *get_hqdate.sh*. Anyway, after some good analysis of what was earlier believed to be an impossible task, it all becomes rather do-able and you can have all the fun doing it the Linux way. Good luck! 

References

- [1] <http://www.gymoire.com/Univ/Sed.html>
- [2] <http://tutorials.jenkov.com/java-regex/matcher.html>

By: Anton Borisov

The author has specialised in Linux and FOSS technologies for more than a decade. His professional spheres of interest include, but are not limited to robotics, embedded systems, statistics and algorithmic methods.